**Institut Teknologi Bandung**

Directorate of Continuing Professional Education

in partnership with

CHAPTER 11

# Image Segmentation

A class for every pixel — built from scratch as an encoder-decoder, then handed to a pretrained model you prompt rather than train.

**Duration** 2.5 hours

**Instructor** Rahman Indra Kesuma, S.Kom., M.Cs.

**Source** Chollet & Watson, Deep Learning with Python 3e — chapter 11

LEARNING OUTCOMES

# Session Objectives

By the end of this session, participants can:

1. Place **classification, segmentation, and detection** — the three tasks almost all computer vision reduces to.
2. Distinguish **semantic, instance, and panoptic** segmentation.
3. Build an **encoder-decoder** segmentation model, and explain why the output must match the input's spatial size.
4. Say why segmentation uses **strided convolutions instead of max pooling**.
5. Explain what `Conv2DTranspose` does and how it undoes a strided convolution.
6. Compute and interpret **Intersection over Union**, and configure the Keras metric correctly for sparse targets.
7. Use **Segment Anything** with point and box prompts, without fine-tuning anything.

BOOK
Chapter 11 — full text

NOTEBOOK
01_segmentation_from_scratch.ipynb

NOTEBOOK
02_conv2dtranspose.ipynb

NOTEBOOK
03_segment_anything.ipynb

REPO
Author's official notebook

SECTION 11.1

# Three tasks almost everything reduces to


Figure 11.1 — classification, segmentation, and detection.

# . . . and the more specialised ones beyond them

But classification, segmentation, and detection are the foundation every engineer should know: **almost all computer vision applications boil down to one of these three**

# Three flavours of segmentation

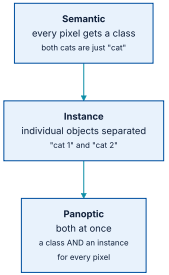



Figure 11.2 — panoptic is the most informative of the three.

All of them assign a class **to each pixel**, partitioning the image into zones — *background* and *foreground*, or *road*, *car*, and *sidewalk*.

SECTION 11.1.1

# Where it is actually used

**Image and video editing**
Background replacement, object removal, compositing.

**Autonomous driving**
Road, lane, and obstacle segmentation at pixel precision.

**Robotics**
Knowing exactly where a graspable object begins and ends.

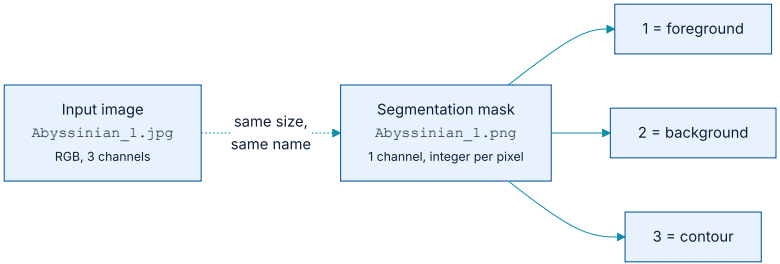
**Medical imaging**
Delineating an organ or a lesion rather than merely detecting it.

# 01

# Training a segmentation model from scratch

An encoder that compresses, and a decoder that puts it back.

# What a label looks like when the label is an image




A segmentation mask is the equivalent of a label: same size as the input, one channel, one integer per pixel.

The dataset is **Oxford-IIIT Pets**: 7,390 pictures of cat and dog breeds, each with a foreground-background mask.

SECTION 11.2.1

# Pairing images with their masks

building the two path lists

PYTHON

```python
import pathlib

input_dir = pathlib.Path("images")
target_dir = pathlib.Path("annotations/trimaps")

input_img_paths = sorted(input_dir.glob("*.jpg"))
target_paths = sorted(target_dir.glob("[!.]*.png"))    # skips spurious dot-files

print(len(input_img_paths), len(target_paths))
```

OUTPUT

```
7390 7390
```

The `[!.]` in the glob is not cosmetic: the trimaps directory contains hidden files, and including them would **silently offset every image-mask pair after the first one**

SECTION 11.2.1

# Shuffling two lists in lockstep

loading into memory PYTHON

```python
img_size = (200, 200)
num_imgs = len(input_img_paths)

random.Random(1337).shuffle(input_img_paths)     # the SAME seed in both calls
random.Random(1337).shuffle(target_paths)        # keeps the pairing intact

def path_to_target(path):
    img = img_to_array(load_img(path, target_size=img_size, color_mode="grayscale"))
    return img.astype("uint8") - 1               # labels 1,2,3 -> 0,1,2

input_imgs = np.zeros((num_imgs,) + img_size + (3,), dtype="float32")
targets = np.zeros((num_imgs,) + img_size + (1,), dtype="uint8")
for i in range(num_imgs):
    input_imgs[i] = path_to_input_image(input_img_paths[i])
    targets[i] = path_to_target(target_paths[i])
```

Two details worth naming: the labels are shifted from **1, 2, 3 to 0, 1, 2** because the loss expects zero-based classes, and the whole dataset is loaded into memory because **7,390 images at 200×200 comfortably fits**

SECTION 11.2.1

# Looking at a mask before training on it

making a mask visible PYTHON

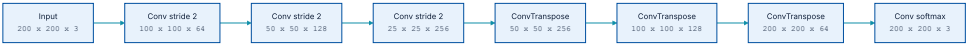
```python
def display_target(target_array):
    # 1,2,3 -> 0,127,254 : black, grey, near-white
    normalized_array = (target_array.astype("uint8") - 1) * 127
    plt.axis("off")
    plt.imshow(normalized_array[:, :, 0])

img = img_to_array(load_img(target_paths[9], color_mode="grayscale"))
display_target(img)
```

The result shows the animal in black, the background in near-white, and a grey contour band between them — **the third class, which is easy to forget exists**

SECTION 11.2.2

# The shape of the problem




Down three times by a factor of two, then back up three times.

The first half is an ordinary classification-style ConvNet: it **compresses** the image into a small feature map where each location carries information about a large chunk of the original.

SECTION 11.2.2

# The model, in one function

the encoder half — PYTHON

```python
from keras.layers import Rescaling, Conv2D, Conv2DTranspose

def get_model(img_size, num_classes):
    inputs = keras.Input(shape=img_size + (3,))
    x = Rescaling(1.0 / 255)(inputs)

    x = Conv2D(64, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2D(64, 3, activation="relu", padding="same")(x)
    x = Conv2D(128, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2D(128, 3, activation="relu", padding="same")(x)
    x = Conv2D(256, 3, strides=2, padding="same", activation="relu")(x)
    x = Conv2D(256, 3, activation="relu", padding="same")(x)
    # ends at (25, 25, 256)
```

Familiar so far — growing filter counts, shrinking spatial size. **Except** that every downsample is a stride rather than a pooling layer.

SECTION 11.2.2

# . . . and the decoder half

the decoder half — PYTHON

```python
    x = Conv2DTranspose(256, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(256, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2DTranspose(128, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(128, 3, strides=2, activation="relu", padding="same")(x)
    x = Conv2DTranspose(64, 3, activation="relu", padding="same")(x)
    x = Conv2DTranspose(64, 3, strides=2, activation="relu", padding="same")(x)

    outputs = Conv2D(num_classes, 3, activation="softmax", padding="same")(x)
    return keras.Model(inputs, outputs)

model = get_model(img_size=img_size, num_classes=3)
```

Three strided convolutions down, three strided transposed convolutions up — **the counts have to match** or the output will not align with the mask.

SECTION 11.2.2

# What that last layer is doing

> The final layer is a **per-pixel softmax**: for every one of the 200×200 output positions it produces a distribution over the three classes. **Classification, repeated 40,000 times.**

Which is why `padding="same"` appears on every layer: any border trimming would leave the output a different size from the mask it is being compared against.

SECTION 11.2.2

# Why no max pooling here


The trade-off that was invisible in chapter 8 becomes decisive here.

SECTION 11.2.2

# The rule that follows from it

So the general rule the book states: **use strides instead of max pooling in any model that cares about feature location** — including **the generative models of chapter 17**

SECTION 11.2.2

# Conv2DTranspose: a convolution that learns to upsample

the two are inverses

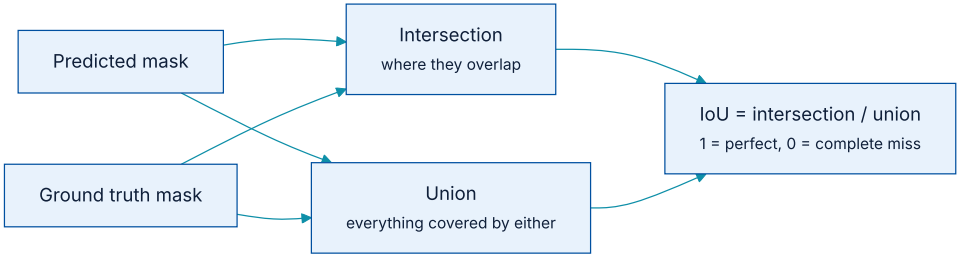
PYTHON

```python
# input (100, 100, 64)
Conv2D(128, 3, strides=2, padding="same")           # -> (50, 50, 128)

# input (50, 50, 128)
Conv2DTranspose(64, 3, strides=2, padding="same")   # -> (100, 100, 64)
```

So a stack of `Conv2D` with strides, mirrored by a stack of `Conv2DTranspose` with the same strides, **returns you to the original spatial size** — which is exactly what the model above does.

# Measuring it: Intersection over Union




Accuracy per pixel would be dominated by the background; IoU is not.

It can be computed per class or averaged across classes, and it is the standard measure of how well a predicted mask matches the ground truth.

SECTION 11.2.2

# Configuring the Keras metric correctly

the IoU metric

PYTHON

```python
foreground_iou = keras.metrics.IoU(
    num_classes=3,
    target_class_ids=(0,),      # which class to score: 0 = foreground
    name="foreground_iou",
    sparse_y_true=True,         # our targets are integer class IDs
    sparse_y_pred=False,        # but our predictions are a dense softmax
)
```

`sparse_y_true=True` with `sparse_y_pred=False` is the combination you almost always want here: **targets are integers, predictions are probabilities**. **Mismatching them does not raise an error.**

SECTION 11.2.2

# Compiling and fitting it

training the segmentation model

PYTHON

```python
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=[foreground_iou])

callbacks = [keras.callbacks.ModelCheckpoint("segmentation.keras",
                                             save_best_only=True)]

history = model.fit(train_dataset, epochs=50,
                    validation_data=validation_dataset, callbacks=callbacks)
```

Everything you learned in chapters 6 and 7 applies unchanged — only the model's output shape and the metric are new.

SECTION 11.2.2

# Splitting, and what the arrays look like

the split

PYTHON

```python
num_val_samples = 1000
train_input_imgs = input_imgs[:-num_val_samples]
train_targets = targets[:-num_val_samples]
val_input_imgs = input_imgs[-num_val_samples:]
val_targets = targets[-num_val_samples:]

print(train_input_imgs.shape, train_targets.shape)
```

OUTPUT

```
(6390, 200, 200, 3) (6390, 200, 200, 1)
```

Note the target shape: **one channel, not three**. The input is RGB; the target is **a single integer per pixel**

SECTION 11.2.2

# Predicting, and reading the output

predicting a mask — PYTHON

```python
model = keras.models.load_model("segmentation.keras")

test_image = val_input_imgs[4]
mask = model.predict(np.expand_dims(test_image, 0))[0]

predicted_mask = np.argmax(mask, axis=-1)      # (200, 200): a class per pixel
print(mask.shape, predicted_mask.shape)
```

OUTPUT

```
(200, 200, 3) (200, 200)
```

That `argmax` is the segmentation equivalent of the `predictions[0].argmax()` from chapter 2 — **performed once per pixel instead of once per image**

# 02

# Using a pretrained segmentation model

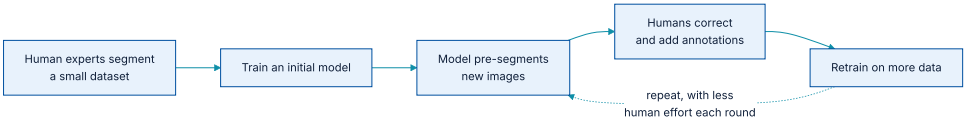
Segment Anything: prompt it, do not train it.

SECTION 11.3

# What SAM is

The main innovation: it is **not limited to a predefined set of object classes**. You segment a new kind of object simply by giving an example of what you are looking for — **and you do not need to fine-tune it first**

# The dataset and the model were built together




The partially trained model was used to help label the data it would later be trained on.

The goal of SA-1B is **fully segmented images**: every object given a unique mask. Each image carries around **100 masks on average**, and some have over **500**.

# Three components, trained together

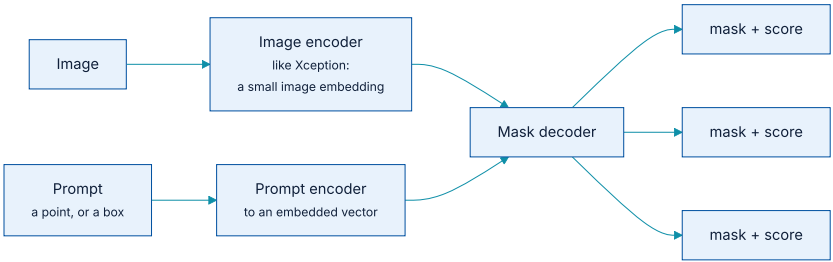



Figure 11.8 — image encoder, prompt encoder, mask decoder. The decoder emits several candidate masks, each with a score.

The image encoder is **something you already know how to build** — it is much like the Xception of chapters 8 and 9. The prompt encoder and mask decoder use techniques from later chapters.

SECTION 11.3.2

# How the training triples are generated

1. For a given image, **choose a random mask** from its annotations.
2. **Randomly choose** whether to create a box prompt or a point prompt.
3. For a **point prompt**, pick a random pixel inside the mask.
4. For a **box prompt**, draw a box around all points inside the mask.

> That can be repeated indefinitely, sampling **many triples from each image** — which is how 11 million images become **a far larger number of training examples**

SECTION 11.3.4

# Two kinds of prompt

**Point prompts**

Select a point and let the model segment the object it belongs to. Points are **labelled**: 1 for foreground, 0 for background.

**Box prompts**

Draw an approximate box around an object — **it does not need to be precise** — and let the model segment what is inside.

In ambiguous cases, pass **several labelled points** rather than one: points labelled 1 say what to include, points labelled 0 say **what to leave out**

SECTION 11.3.4

# Prompting it with a single point

a point prompt — PYTHON

```python
import numpy as np

input_point = np.array([[580, 450]])     # coordinates of the point
input_label = np.array([1])              # 1 = foreground, 0 = background

outputs = model.predict({
    "images": ops.expand_dims(image, axis=0),
    "points": ops.expand_dims(input_point, axis=0),
    "labels": ops.expand_dims(input_label, axis=0),
})
```

> One point, no training, no class list — and SAM returns a mask for the peach that point landed on. **That is the shift this section is about.**

SECTION 11.3.4

# Reading the output

picking a mask

```python
masks = outputs["masks"][0]
scores = outputs["iou_pred"][0]

best = int(np.argmax(scores))
mask = masks[best]
print(mask.shape, float(scores[best]))
```

Taking the top-scoring mask is the default, but the ambiguity is real information: **if the three candidates disagree wildly, the prompt itself was ambiguous** — and **an extra background point usually resolves it**

SECTION 11.3

# From scratch, or pretrained?

| | FROM SCRATCH (§11.2) | SEGMENT ANYTHING (§11.3) |
|---|---|---|
| **What you need** | A labelled mask for every training image. | One point or box, at inference time. |
| **Classes** | Fixed at training time. | **None** — it segments whatever you point at. |
| **Training cost** | A full training run. | Zero. |
| **When it wins** | You need a **specific, repeatable** class set — medical structures, industrial defects — and you have the masks. | You need **generic** object masks, or you have no labelled data at all. |

SECTION 11.3.1

# Loading SAM

instantiating Segment Anything

PYTHON

```python
import keras_hub

model = keras_hub.models.SAMImageSegmenter.from_preset("sam_huge_sa1b")

path = keras.utils.get_file(
    origin="https://s3.amazonaws.com/keras.io/img/book/fruits.jpg")
image = np.array(keras.utils.load_img(path))
```

No training data, no class list, no fine-tuning step — **the next thing that happens is a prediction**

SECTION 11.3

# Why a promptable model changes the workflow

**The old shape**

Decide your classes → collect masks for each → train → the model can segment **those classes and no others**.

**The new shape**

Point at the thing → get a mask. The class set is **decided at inference time**, by the person using it.

That is the same move as prompting a language model, arriving in vision — and it is why chapter 1 grouped **foundation models** as a category rather than a technique.

SECTION 11.3.4

# Refining an ambiguous prompt

adding a background point

PYTHON

```python
input_point = np.array([[580, 450],      # on the peach   -> include
                        [300, 800]])     # on the table   -> exclude
input_label = np.array([1, 0])           # 1 = foreground, 0 = background

outputs = model.predict({
    "images": ops.expand_dims(image, axis=0),
    "points": ops.expand_dims(input_point, axis=0),
    "labels": ops.expand_dims(input_label, axis=0),
})
```

> This is the practical loop with SAM: **prompt, look, add a point, prompt again**. It is closer to **using a selection tool** than to training a model.

PRACTICE

# Where segmentation quietly goes wrong

**Images and masks drift apart**

Two lists sorted or shuffled differently. Nothing errors — the model simply learns nothing, and the loss curve looks like a capacity problem.

**Off-by-one labels**

Masks numbered 1..N fed to a loss expecting 0..N-1. The last class is never predicted and the first is never learned.

**Accuracy instead of IoU**

If 90% of pixels are background, a model predicting *background* everywhere scores 90% accuracy and **zero** foreground IoU.

**Padding mismatch**

One layer without `padding="same"` and the output no longer matches the mask's size.

SUMMARY

# What has to survive this chapter

1. **Classification, segmentation, detection** — almost every vision application is one of these three.
2. **Semantic, instance, panoptic** — increasing amounts of information per pixel.
3. A segmentation model is an **encoder-decoder**: compress, then restore the original spatial size.
4. **Strides, not max pooling**, whenever location matters — pooling destroys position within the window.
5. `Conv2DTranspose` **learns to upsample**, undoing a strided convolution.
6. **IoU, not accuracy** — and check `sparse_y_true` / `sparse_y_pred` against your data.

SUMMARY

# . . . and the shift that SAM represents

A pretrained model that is **prompted rather than trained**, with no predefined class list. It is the first model in this book that works that way — and **the pattern returns in every chapter from 15 onwards**

NOTEBOOK

03_segment_anything.ipynb

NEXT

Chapter 12 — Object detection